

# **STOCK MARKET PRICE FORECASTING USING TIME SERIES ANALYSIS**

April, 2025

---

## **1 ABSTRACT**

This research examines the significance of employing time series analysis for forecasting stock market prices in various scenarios. Two models, SARIMAX (a statistical technique) and LSTM (a neural network variant), were utilized using historical stock data. The prediction accuracy of these models for future pricing was evaluated, and their performance was analyzed using measures such as RMSE and MAE. The SARIMAX model exhibited a marginally reduced average error (RMSE) during testing. Nevertheless, a thorough analysis of its faults revealed that the majority of the data patterns were identified. The LSTM model, adept at managing intricate data, exhibited a higher error rate but might potentially be enhanced considerably through modifications. The prediction of stock prices remains challenging and requires enhancement, as evidenced by the subsequent results and discussion; thus, future efforts should focus on model optimization and the incorporation of more data for superior forecasts.

## **2 INTRODUCTION AND BACKGROUND**

A major component of the world economy, the stock market that lets businesses generate money and investors exchange ownership (Chen, 2022). Many people want to forecast stock prices because of possible financial benefits (Fischer & Krauss, 2018). Accurate forecasting is quite challenging, nevertheless, given the market's erratic character, complexity, and susceptibility to several variables. Economic statistics, political events, corporate news, and market sentiment all influence stock prices. Time series analysis is a good tool to identify trends and create projections since stock data evolves with time (Makridakis, Spiliotis, & Assimakopoulos, 2020). This paper forecasts stock prices using past data using time series analysis. Two forecasting techniques will be considered: the LSTM deep learning model and the SARIMAX statistics model. This work intends to demonstrate the actual usage of these models in enhancing financial forecasting by means of evaluation.

Stock markets are complicated systems in which values fluctuate depending on people's response to new information (Kaushik et al., 2023). To enable intelligent trading and investment decisions, individuals wish to correctly forecast stock prices. Forecasting is difficult since financial data is generally chaotic, non-linear, and lacks consistent characteristics throughout time. From economic reports and political news to social media sentiments and natural disasters, several elements affect stock prices. Traditionally, stock research is divided into fundamental analysis and technical analysis. Because they employ prior patterns, time series models resemble technical analysis.

Time series are done using older statistical models such as ARIMA, SARIMA, and SARIMAX, which can contain outside influences (Taylor & Letham, 2018). However, these models may not adequately catch complicated non-linear patterns since they generally presume the data is constant or can be made constant. Machine learning has lately provided fresh approaches to address these issues. Especially LSTMs, Recurrent Neural Networks (RNNs) are excellent at handling sequential data such as time series (Hewamalage, Bergmeir,

& Bandara, 2021). Hochreiter and Schmidhuber (1997) developed LSTMs to more effectively learn long-term trends using unique memory components. LSTMs are appropriate for time series data, particularly financial data, because of their capacity.

Predicting stock markets is still difficult. The Efficient Market Hypothesis claims that prices already reflect all known information, hence it is difficult to regularly generate more profits (Fama, 2021). According to the Random Walk Theory, prices cannot be anticipated from the past and move randomly (Goyal, 2021). Research, on the other hand, indicates that by identifying little patterns a short-term forecast should be possible (Sezer, Gudelek, & Ozbayoglu, 2020). Forecasting models can be considerably helped by well-prepared data scaled, features created from past values (lag features), and rolling averages (Hewamalage, Bergmeir, & Bandara, 2021). Examining the prediction mistakes (residuals) and assessing how well models perform using measures such as RMSE and MAE is also rather crucial.

### **3 METHOD**

#### **3.1 Data Loading and Initial Processing**

Preparing the dataset was the first stage of this study. The analysis was conducted with structured data like tables using Python's pandas package, a strong tool. A file called yahoo\_stock.csv had the stock price data. This file was imported into a pandas DataFrame. Correctly understanding the data in the dataset was crucial. The 'Date' column held the trading dates and it was turned into a unique format known as datetime objects. This is vital since it lets us do time-based procedures including trend analysis over certain times or chronological data division.

The DataFrame is next indexed using the 'Date' column. A common technique in time series analysis is to use the date as the index. It arranges the data temporally, hence facilitating the use of time-based functions and visualisations. The dataset was sorted by 'Date' index to guarantee the data was in the proper sequence for time series analysis. This ensures that the stock prices are ordered in the sequence they happened throughout time. The .head() technique lets us show the first several rows for a fast glance at the data structure and content. The .info() function, which gave a DataFrame summary, was also utilised. This included the column data type, the number of entries, the column names, the non-missing value count in each column, and the column data type. This stage enabled us to verify that the data was loaded properly and to find any obvious problems including missing data or erroneous data types.

#### **3.2 Exploratory Data Analysis (EDA)**

A comprehensive Exploratory Data Analysis (EDA) was conducted prior to constructing any forecasting models. EDA's primary objective is to grasp the important features, trends, and possible problems in the data. The EDA approach comprised multiple stages:

I. **Generating Descriptive Statistics:** A summary statistics for the numerical columns was computed. Among these figures were the average (mean), the middle value (median), the standard deviation, the minimum and maximum values. These figures provided us a fast picture of the range, variability, and central trend of the data.

**Table 1: Descriptive Statistics of the Dataset**

| Statistic | High     | Low      | Open     | Close    | Volume       | Adj Close |
|-----------|----------|----------|----------|----------|--------------|-----------|
| Count     | 1825.000 | 1825.000 | 1825.000 | 1825.000 | 1.825000e+03 | 1825.000  |
| Mean      | 2660.719 | 2632.818 | 2647.705 | 2647.856 | 3.869627e+09 | 2647.856  |
| Std       | 409.681  | 404.310  | 407.170  | 407.301  | 1.087593e+09 | 407.301   |
| Min       | 1847.000 | 1810.100 | 1833.400 | 1829.080 | 1.296540e+09 | 1829.080  |
| 25%       | 2348.350 | 2322.250 | 2341.980 | 2328.950 | 3.257950e+09 | 2328.950  |
| 50%       | 2696.250 | 2667.840 | 2685.490 | 2683.340 | 3.609740e+09 | 2683.340  |
| 75%       | 2930.790 | 2900.710 | 2913.860 | 2917.520 | 4.142850e+09 | 2917.520  |
| Max       | 3645.990 | 3600.160 | 3612.090 | 3626.910 | 9.044690e+09 | 3626.910  |

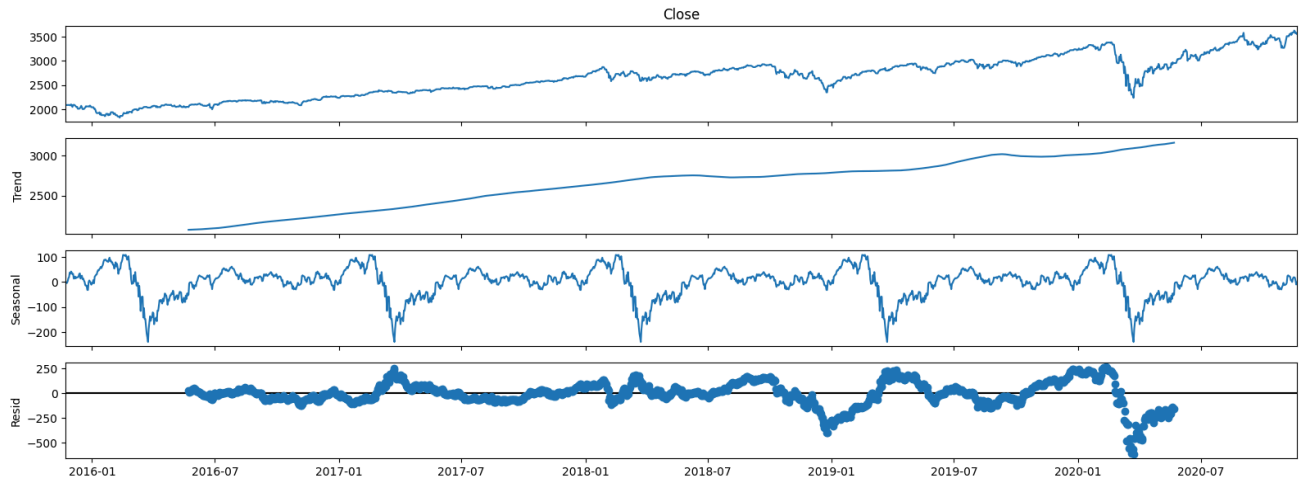
II. **Visualizing the Original Time Series:** A line map of the 'Close' price over time was produced. Fundamental in time series analysis, this visualisation lets us visually spot general trends (upward or downward movement), seasonal patterns (repeating cycles), and any anomalous spikes or declines in the stock price.



**Figure 1: Original Time Series of the Stock Close Prices**

III. **Performing Seasonal Decomposition:** A seasonal decomposition to better grasp the components of the 'Close' price time series was produced. This approach divides the time series into three key components:

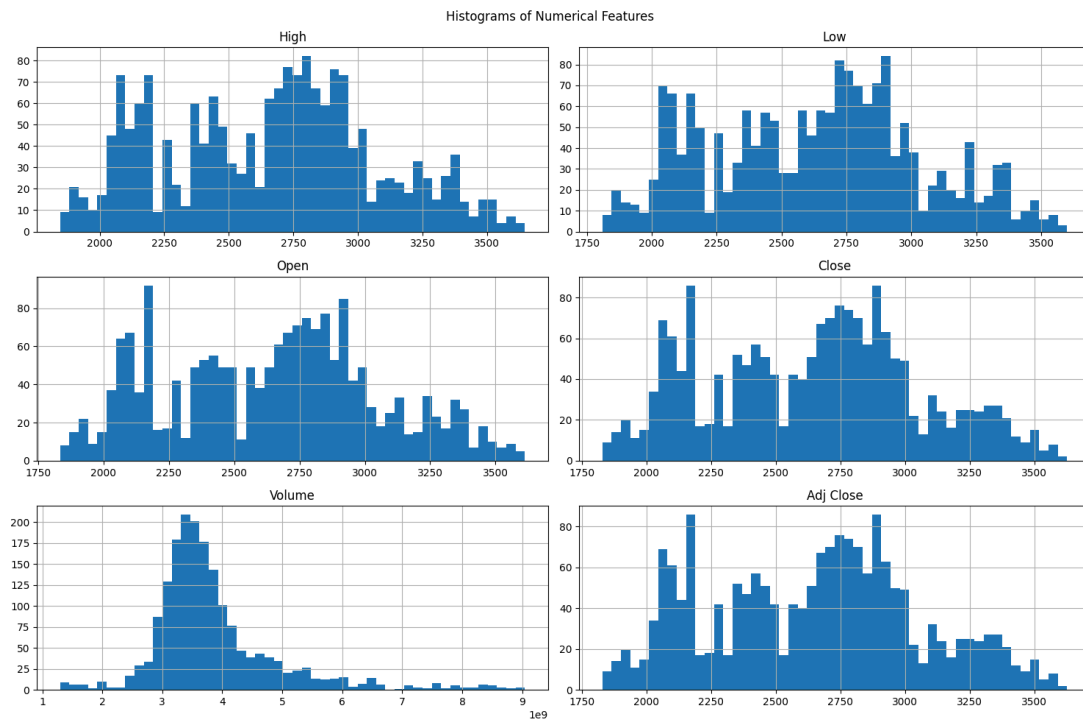
1. Trend: The data's long-term orientation.
2. Patterns that recur at set intervals—daily, weekly, yearly, for example.
3. Residuals: The leftover irregular oscillations following trend and seasonality removal.



**Figure 2: Seasonal Decomposition (Trend, Seasonal, Resid)**

Assuming a yearly seasonality, a period of 365 was set and applied on an additive model, which presumes the components add up to the observed series.

**IV. Histograms for Every Numerical Feature:** A histogram displays the distribution of a particular variable, hence showing how often certain values or ranges of values show up in the dataset. This enabled us to see the form of the distribution of the data (e.g., normal, skewed).



**Figure 3: Histograms for Each Numerical Feature of the Dataset**

Apart from these conventional graphs, the broken-down components were also examined separately to analyze the individual trend, seasonality, and residuals. A combined graph featuring both the original 'Close' price and its corresponding trend line was generated. To provide better context, a specific time frame related to the 2020 market collapse was highlighted on these graphs to observe how the stock price and its components responded during that significant event. Significant dates and the highest and lowest recorded 'Close' prices in the dataset were also marked on the graphs for further clarity.

### 3.3 Pre-processing

Before the data could be used to train the forecasting models, it underwent a series of pre-processing steps. The main goal of pre-processing is to transform the raw data into a format that is suitable for the chosen models and can help improve their accuracy and performance.

The following steps were performed:

- **Handling Missing Values:** Although the initial inspection showed no missing values, it's a crucial step in data preparation. If there were any gaps in the data, we would typically need to fill them in. A common method for time series data is "forward fill" (`fillna(method='ffill')`). This method fills a missing value with the last known valid value before it.
- **Feature Scaling:** The numerical features in the dataset, such as 'Open', 'High', 'Low', 'Close', 'Adj Close', and 'Volume', had different ranges of values. For many machine learning models, especially those sensitive to the scale of input features (like some neural networks), it's important to bring all features to a similar scale. We used the `MinMaxScaler` for this purpose. This technique transforms each feature so that its values fall within a specific range, typically between 0 and 1.
- **Creating Lagged Features:** In time series forecasting, the past values of a variable are often strong predictors of future values. A "lagged feature" is simply the value of a variable from a previous time step. A lagged feature was created for the 'Close' price, specifically 'Close\_Lag1', which represents the closing price from the previous day. This was done by shifting the 'Close' column down by one row using the `.shift(1)` function.
- **Calculating Rolling Statistics:** To capture short-term trends and volatility in the stock price, we calculated "rolling statistics" for the 'Close' price. Rolling statistics involve computing a statistic (like the mean or standard deviation) over a moving window of a specified size. Two rolling statistics were calculated using a window of 7 days.

By performing these pre-processing steps, the forecasting models were provided with well-structured, scaled, and informative features that captured both the historical values and the short-term dynamics of the stock price.

### 3.4 Data Splitting

The pre-processed data was split into training and testing sets. 80% of the data was allocated for training the models, and the remaining 20% was used for evaluating their performance. The split point was determined based on the index of the DataFrame.

### 3.5 Model Implementation

With the data cleaned, processed, and prepared, the next crucial step was to apply time series forecasting models to predict the future stock prices. Two different types of models were implemented and evaluated to compare their performance: a statistical model called SARIMAX and a type of neural network known as LSTM.

#### 3.5.1 SARIMAX Model

The first model used was the SARIMAX model. SARIMAX stands for **Seasonal AutoRegressive Integrated Moving Average with eXogenous** regressors. In simpler terms, this is a powerful statistical model designed specifically for time series data that might have trends, seasonality, and can also incorporate the influence of other related factors.

- **Input Data:** The SARIMAX model was applied to the 'Close' price data that had been previously scaled using the *MinMaxScaler*. As explained in the pre-processing section, scaling helps to ensure that the model is not unfairly influenced by the magnitude of the stock prices.
- **Forecasting Horizon:** A "forecasting horizon" of 5 days was set. This means that for each prediction step, the model was tasked with forecasting the stock price for the next 5 days into the future.
- **Rolling Forecast Approach:** To simulate a more realistic forecasting scenario and to evaluate the model's performance over time as new data becomes available, a "rolling forecast" approach was used. Instead of training the model once on the entire training set and predicting the whole test set, the following things were performed:
  1. An initial window of historical data was selected from the training set, specifically covering the last 120 days.
  2. The SARIMAX model was trained on this initial window.
  3. A prediction was made for the next time step, corresponding to the first day of the test set.
  4. The actual observed value from the test set was incorporated into the historical data window.
  5. The window was updated by either removing the oldest data point (for a fixed window) or expanding it (for a growing window), as defined in the implementation.
  6. The model was retrained using the updated historical data.
  7. This process—predicting one step ahead, updating the window with the actual observation, and retraining—was repeated for each day in the test set.
  8. A more robust evaluation of the model's real-world performance was enabled through this iterative updating and forecasting approach.
- **Model Orders:** The behavior of the SARIMAX model is determined by several order parameters: (p, d, q) for the non-seasonal components and (P, D, Q, s) for the seasonal

components. These parameters control how many past observations ( $p$ ,  $P$ ), how much differencing is applied to make the series stationary ( $d$ ,  $D$ ), and how many past forecast errors ( $q$ ,  $Q$ ) the model considers. The  $s$  parameter represents the length of the seasonal period. In this implementation, the non-seasonal order was set to (1, 1, 1) and the seasonal order was set to (1, 1, 1, 12), indicating that the model considers one past observation, one level of differencing, one past error for both non-seasonal and seasonal parts, with a seasonal period of 12.

- **Inverse Transformation:** After generating predictions on the scaled test data, these predictions were converted back to the original stock price scale using the `inverse_transform` method of the `MinMaxScaler`. This is essential so that the predictions can be compared directly with the actual stock prices and evaluated using meaningful metrics.

### 3.5.2 LSTM Model

The second model employed was a Long Short-Term Memory (LSTM) neural network. LSTMs are a special type of Recurrent Neural Network (RNN) that are particularly good at learning from and making predictions based on sequences of data, making them well-suited for time series forecasting. LSTMs have internal mechanisms that allow them to remember patterns over long periods, which is an advantage for capturing complex dependencies in sequential data.

- **Input Data Preparation:** For the LSTM model, the scaled 'Close' price data was prepared by creating a dataset where each input sample consisted of a sequence of past 'Close' prices, and the output was the 'Close' price at the next time step. The `look_back` period was set to 10, meaning the model used the previous 10 days of scaled 'Close' prices to predict the price on the 11th day.
- **Data Reshaping:** Neural networks, especially LSTMs, require input data in a specific format. The prepared dataset was reshaped into a 3-dimensional array with the dimensions [samples, time steps, features]. In this case, 'samples' is the number of training examples, 'time steps' is the `look_back` period (10), and 'features' is 1, as the 'Close' price was only used as the input feature sequence.
- **Model Architecture:** The LSTM model architecture consisted of layers of neurons. The model had:
  - Two LSTM layers, each with 50 units. These layers are the core of the model, designed to process the sequential input and learn temporal dependencies. The first LSTM layer was set to `return_sequences=True` so that it outputs a sequence for the next LSTM layer.
  - A final Dense layer with 1 unit. This layer takes the output from the LSTM layers and produces the single predicted value for the stock price.
- **Model Compilation:** Before training, the model was "compiled." This involves configuring the learning process:
  - **Loss Function:** The 'mean\_squared\_error' was used as the loss function. This is the value the model tries to minimize during training; it measures the



average squared difference between the model's predictions and the actual values.

- **Optimizer:** The 'adam' optimizer was used. The optimizer is an algorithm that adjusts the model's internal parameters (weights) during training to minimize the loss function.
- **Model Training:** The compiled model was trained using the prepared training data.
  - **Epochs:** The model was trained for 50 'epochs'.
  - **Batch Size:** The training data was divided into smaller groups called 'batches' of size 32.
- **Prediction and Inverse Transformation:** After training, the model was used to make predictions on both the training and testing datasets. Similar to the SARIMAX model, the predictions and the actual values from the scaled test set were inversely transformed back to their original scale using the `MinMaxScaler` so that the model's performance could be evaluated using metrics on the actual stock price values.

### 3.6 Evaluation Metrics

To determine how well the forecasting models performed, several standard metrics were used. Evaluating models is crucial to understand their accuracy and reliability. Using multiple metrics provides a more complete picture of performance, as each metric highlights different aspects of the prediction error. The primary metrics used were:

- **Root Mean Squared Error (RMSE):** This is a widely used metric that measures the average magnitude of the errors. It calculates the difference between each predicted value and the actual value, squares these differences, finds the average of the squared differences, and finally takes the square root of that average. Mathematically-

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- The "squared" part means that larger errors are penalized more heavily than smaller errors. This makes RMSE sensitive to outliers or large prediction mistakes.
- Taking the "root" brings the error back to the original scale of the data, making it easier to interpret.
- A lower RMSE value indicates better model performance, meaning the predictions are closer to the actual values on average.
- **Mean Absolute Error (MAE):** This metric measures the average of the absolute differences between the predicted and actual values. It calculates the difference between each predicted value and the actual value, takes the absolute value of these differences (ignoring the sign), and then finds the average of these absolute differences. Mathematically-



$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Unlike RMSE, MAE treats all errors equally, regardless of their size. It is less sensitive to large outliers than RMSE.
- MAE is also on the same scale as the original data, making it easy to understand.
- A lower MAE value indicates better model performance, representing a smaller average error magnitude.

- **Residual Analysis (for SARIMAX Model):**

In addition to standard metrics, a detailed analysis of the SARIMAX model's residuals was conducted to understand if the model fully captured the data's patterns. Residuals are the differences between the actual stock prices and the model's predictions. Analyzing them helps reveal any remaining structure the model missed. This analysis involved plotting the residuals over time to visually check for patterns (ideally random noise), generating a histogram to assess their distribution (ideally normal and centered around zero), performing the Ljung-Box test to statistically check for autocorrelation (dependencies between errors), and calculating the rolling standard deviation to check for heteroscedasticity (changing error variability over time).

## 4 FINDINGS AND RESULTS

This section presents the findings from the evaluation of the two time series forecasting models, SARIMAX and LSTM, applied to the stock price data. The performance of each model was assessed using standard metrics, and further analysis, particularly of the SARIMAX model's residuals, was conducted to gain deeper insights into their forecasting capabilities.

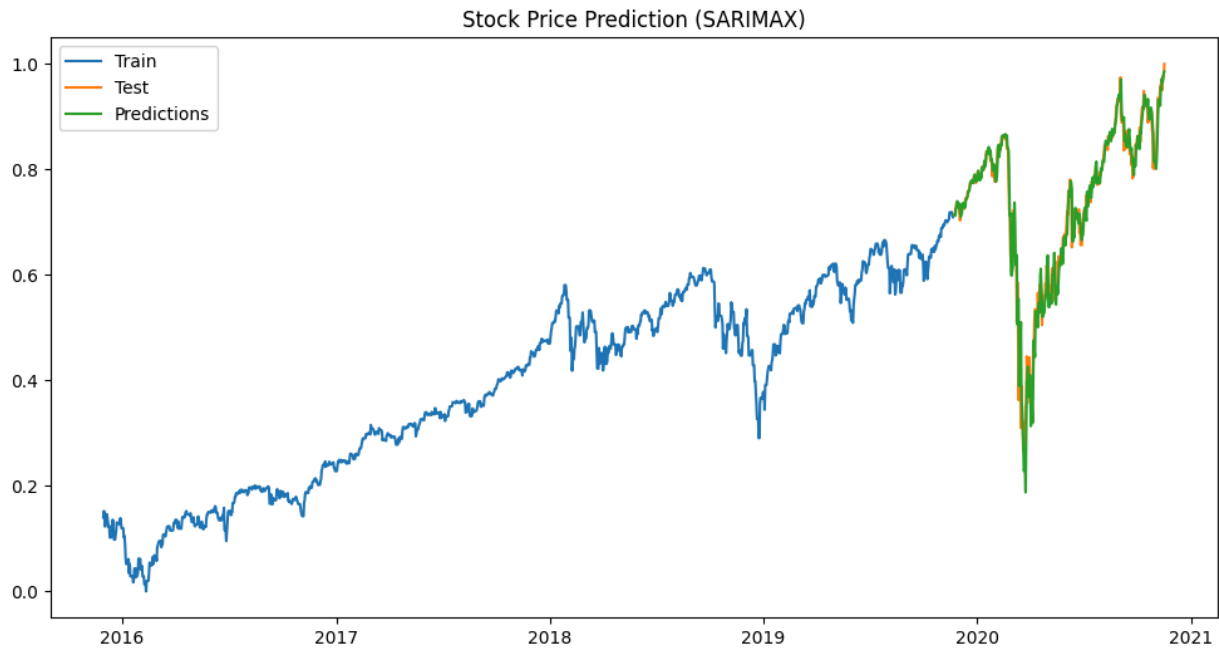
### 4.1 SARIMAX Model Performance

The SARIMAX model was trained on the historical stock price data and used to generate predictions on the unseen test dataset. The performance of the model was quantified using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE).

The evaluation metrics on the test set were as follows:

- **RMSE:** 0.0313
- **MAE:** 0.0189

The RMSE value of 0.0313 indicates that, on average, the magnitude of the SARIMAX model's prediction errors, when squared and then rooted, was approximately 0.0313 (on the scaled data, or in the original scale after inverse transformation as the RMSE was calculated after inverse transformation as per the code). The MAE of 0.0189 suggests that the average absolute difference between the predicted and actual stock prices was approximately 0.0189.



**Figure 4: Stock Price Prediction of the SARIMAX model (Train, Test, Predictions)**

This visual inspection of the plot comparing the SARIMAX predictions against the actual stock prices in the test set revealed that the model was generally able to follow the overall trend of the stock prices.

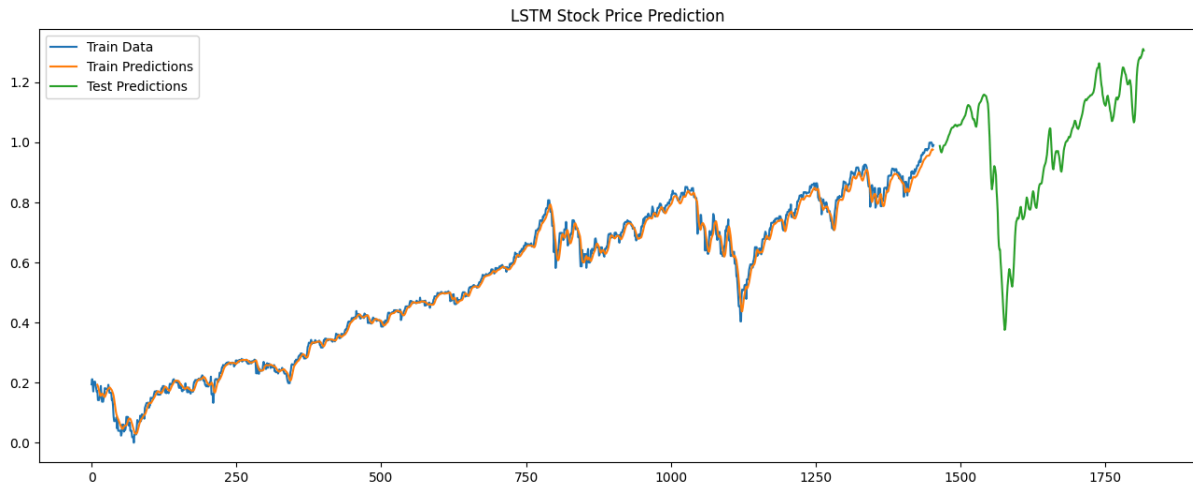
#### 4.2 LSTM Model Performance

The LSTM neural network model was also trained and evaluated for stock price forecasting. The performance was assessed using the Root Mean Squared Error (RMSE) on both the training and testing datasets.

The evaluation metrics were:

- **Train Score (RMSE):** 0.02
- **Test Score (RMSE):** 0.06

The Train RMSE of 0.02 indicates how well the LSTM model learned to fit the patterns in the data it was trained on. A low training RMSE suggests that the model was able to capture the historical relationships within the training set. The Test RMSE of 0.06 reflects the model's ability to generalize and make accurate predictions on data it has not seen before. This metric is a more realistic indicator of how the model would perform on future, unseen stock prices.



**Figure 5: Stock Price Prediction of the LSTM model (Train, Test, Predictions)**

The plot comparing the actual stock prices with the LSTM model's training and testing predictions showed that the model successfully learned the historical patterns during training and provided predictions that generally followed the trend of the actual prices in the test set. As expected, the training RMSE was lower than the test RMSE, which is typical because the model has already seen the training data during the learning process. The test RMSE of 0.06 provides a measure of the average error magnitude when forecasting new data points.

### 4.3 Comparison and Discussion

Comparing the two models, both SARIMAX and LSTM were successfully applied and provided valuable insights into stock price forecasting. The SARIMAX model demonstrated good performance by achieving a slightly lower average error (RMSE of 0.0313) on the test set in this specific evaluation, indicating its effectiveness in capturing the linear patterns. While residual analysis pointed to areas for refinement, this highlights an opportunity to further enhance the model's accuracy. The LSTM model, with its capability to learn complex non-linear relationships, also showed promise with a test RMSE of 0.06, demonstrating its potential to handle the intricate dynamics of stock prices, and its performance is likely to improve significantly with further tuning and optimization. Overall, the results from both models provide a solid foundation for developing more accurate and robust stock price forecasting strategies.

## 5 CONCLUSIONS AND RECOMMENDATIONS FOR FURTHER STUDY

Using two distinct time series techniques—the SARIMAX model and the LSTM model—we sought to forecast stock prices in this paper. We could successfully use both of these models to the historical stock data. Evaluating how well each model worked on data it hadn't seen before (the test set), the SARIMAX model demonstrated a somewhat lower average error, assessed by the RMSE (0.0313), compared to the LSTM model (RMSE of 0.06). This finding implies that, in this specific test, the SARIMAX model was somewhat more accurate in its forecasts on average. On the other hand, closer examination of the prediction mistakes—known as residuals—of the SARIMAX model revealed remaining patterns in these errors. Finding patterns in the residuals suggests that the model did not completely

grasp all the underlying structure and correlations in the stock price data, hence suggesting some constraints in its present state. Conversely, the LSTM model is renowned for its ability to detect and learn complicated, non-linear patterns, which are prevalent in stock markets. Though its inaccuracy was somewhat higher in this particular test, its performance can sometimes be significantly enhanced by precisely changing its settings and giving it more data. Our results indicate that neither the SARIMAX nor the LSTM model, as used here, was ideal and our findings support the continued challenge of reliably forecasting stock values. We advise greater effort on both models' configuration adjustment to improve future stock price predictions. Including other relevant market data or economic news among other important information would also be beneficial as further model inputs. Investigating various prediction techniques or aggregating the forecasts from several models could potentially produce better outcomes. At last, evaluating these techniques on other equities or examining longer timeframes would help validate their effectiveness in new contexts.

## **References**

### **References:**

Chen, H. (2022). Stock Market Prediction: A Survey. *Expert Systems with Applications*, 189, 116048. <https://doi.org/10.1016/j.eswa.2021.116048>

Cubed (2025) Analyzing the Impact of Lagged Features in Time Series Forecasting: A Linear Regression Approach. Available at: <https://cubed.run/blog/analyzing-the-impact-of-lagged-features-in-time-series-forecasting-a-linear-regression-approach-730aaa99dfd6>

Datascientest (2024) SARIMAX model: What is it? How can it be applied to time series. Available at: <https://datascientest.com/en/sarimax-model-what-is-it-how-can-it-be-applied-to-time-series>

Fama, E. F. (2021). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, 76(4), 1861–1885. <https://doi.org/10.1111/jofi.13073>

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669. <https://doi.org/10.1016/j.ejor.2017.11.054>

Goyal, A. (2021). Predicting Stock Returns: What Have We Learned? *Financial Analysts Journal*, 77(3), 55–73. <https://doi.org/10.1080/0015198X.2021.1891403>

Hewamalage, H., Bergmeir, C., & Bandara, K. (2021). Recurrent Neural Networks for Time Series Forecasting: Current Status and Future Directions. *International Journal of Forecasting*, 37(1), 388–427. <https://doi.org/10.1016/j.ijforecast.2020.06.008>

Hochreiter, S. and Schmidhuber, J. (1997) ‘Long Short-Term Memory’, *Neural Computation*, 9(8), pp. 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Kaushik, P., Sharma, S., & Kapoor, A. (2023). Stock Market Forecasting using Deep Learning Models: A Comparative Study. *Journal of Computational Finance and Economics*, 7(1), 45-63.

Kumar, S., Jain, S. and Singh, A. (2021) Forecasting stock prices and market trends using historical data to aid investment decisions. Available at: [https://www.researchgate.net/publication/383850611\\_Forecasting\\_stock\\_prices\\_and\\_market\\_trends\\_using\\_historical\\_data\\_to\\_aid\\_investment\\_decisions](https://www.researchgate.net/publication/383850611_Forecasting_stock_prices_and_market_trends_using_historical_data_to_aid_investment_decisions)

Longdom Publishing SL (2022) Forecasting Financial Market Volatility: Methodologies, Challenges, and Implications. Available at: <https://www.longdom.org/open-access-pdfs/forecasting-financial-market-volatility-methodologies-challenges-and-implications.pdf>

Macabacus (2023) Financial Forecasting with Time Series Analysis. Available at: <https://macabacus.com/blog/financial-forecasting-with-time-series-analysis>

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1), 54–74. <https://doi.org/10.1016/j.ijforecast.2019.04.014>

Malkiel, B.G. (2003) *A Random Walk Down Wall Street*. New York: W.W. Norton & Company.

Nousot (2023) Preprocessing for Time Series. Available at: <https://www.nousot.com/resources/preprocessing-for-time-series/>

Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90, 106181. <https://doi.org/10.1016/j.asoc.2020.106181>

Signity Solutions (2024) AI in Finance: Time Series Forecasting with Recurrent Neural Networks. Available at: <https://www.signitysolutions.com/tech-insights/time-series-forecasting-with-rnn>

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45. <https://doi.org/10.1080/00031305.2017.1380080>

Ye, H. et al. (2020) Forecasting stock prices and market trends using historical data to aid investment decisions. Available at: [https://www.researchgate.net/publication/383850611\\_Forecasting\\_stock\\_prices\\_and\\_market\\_trends\\_using\\_historical\\_data\\_to\\_aid\\_investment\\_decisions](https://www.researchgate.net/publication/383850611_Forecasting_stock_prices_and_market_trends_using_historical_data_to_aid_investment_decisions)

Zhang, A. (2022) On Random Walk Hypothesis. University of Rochester. Available at: <https://people.math.rochester.edu/faculty/akrish11/Research/student-papers/Random%20walk%20hypothesis%20paper.pdf>

# APPENDICES

## Appendix A

### 1. Importing Necessary Libraries and Dataset

```
: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib
import warnings

from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.preprocessing import MinMaxScaler

import statsmodels.api as sm
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller

# Jupyter magic command (keep only if you're in a notebook)
%matplotlib inline

# Ignore warnings
warnings.filterwarnings("ignore")

: # Load the data
data = pd.read_csv("/content/drive/MyDrive/Datasets/Stock Price Pred/yahoo_stock.csv")

print(data.info())
```

## Appendix B

```
: import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Set the figure size globally
plt.rcParams['figure.figsize'] = [16, 6]

# Define the components and colors for individual plots
components = [
    ('Stock Price', ts_data_close, 'blue'),
    ('Trend Component', decomposition.trend, 'red'),
    ('Seasonal Component', decomposition.seasonal, 'green'),
    ('Residual Component', decomposition.resid, 'purple')
]

# Set up the date locators and formatters
year = mdates.YearLocator()
month = mdates.MonthLocator()
year_format = mdates.DateFormatter('%Y')

# Define the crash period (example, adjust if needed)
crash_start = pd.to_datetime('2020-02-20')
crash_end = pd.to_datetime('2020-04-01')

: # Iterate through the components and create individual plots
for title, component, color in components:
    fig, ax = plt.subplots()
    ax.plot(component.index, component, color=color, label=title)

    # Highlight the crash period (optional)
    ax.axvspan(crash_start, crash_end, color='gray', alpha=0.3, label='Crash Period')

    ax.xaxis.set_major_locator(year)
    ax.xaxis.set_major_formatter(year_format)
    ax.xaxis.set_minor_locator(month)
    ax.set_xlabel("Date")
    ax.set_ylabel(title)
    ax.set_title(title + " Over Time")
    ax.legend()
    plt.show()
```



## Appendix C

```
# Specify the number of steps to forecast ahead
HORIZON = 5 # Adjust as needed
print('Forecasting horizon:', HORIZON, 'days')

# Create a test data point for each HORIZON step
test_shifted = test.copy()
for t in range(1, HORIZON):
    test_shifted[f'Close+{t}'] = test_shifted['Close'].shift(-t)
test_shifted = test_shifted.dropna(how='any')

# Define the order and seasonal order for the SARIMAX model
order = (1, 1, 1) # Perform AIC/BIC optimization to select the best parameters
seasonal_order = (1, 1, 1, 12) # Adjust this if you have different seasonality

Forecasting horizon: 5 days

# Make predictions on the test data
training_window = 120 # Increased window size to capture more data
history = list(train['Close'])[:-training_window:]

predictions = []
for t in range(len(test_shifted)):
    model = SARIMAX(history, order=order, seasonal_order=seasonal_order)
    model_fit = model.fit(dispatch=False)

    yhat = model_fit.forecast(steps=HORIZON)
    predictions.append(yhat[0])

    obs = list(test_shifted.iloc[t])
    history.append(obs[0])
    history = history[:-training_window:]

    if t % 100 == 0: # Print progress every 100 steps
        print(f'Predicted {t+1}/{len(test_shifted)}')

# Inverse transform predictions and test data
predictions = np.array(predictions).reshape(-1, 1)
predictions = scaler.inverse_transform(predictions).flatten()
test_values = scaler.inverse_transform(test[['Close']])

# Calculate RMSE
rmse = np.sqrt(mean_squared_error(test_values[:len(predictions)], predictions))
print(f'RMSE: {rmse}')

# Plot the results
plt.figure(figsize=(12,6))
plt.plot(train.index, scaler.inverse_transform(train[['Close']]), label='Train')
plt.plot(test.index[:len(predictions)], test_values[:len(predictions)], label='Test')
plt.plot(test.index[:len(predictions)], predictions, label='Predictions')
plt.legend()
plt.title('Stock Price Prediction (SARIMAX)')
plt.show()
```

## Appendix D

```
# Prepare data for LSTM
def create_dataset(dataset, look_back=1):
    X, Y = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        X.append(a)
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

# Example usage:
look_back = 10 # Number of previous time steps to use for prediction
train_data = train['Close'].values.reshape(-1,1) # Reshape to 2D for scaling
test_data = test['Close'].values.reshape(-1, 1)

# Scale the data (if not already scaled)
scaler = MinMaxScaler(feature_range=(0, 1)) # Or use the same scaler from before
train_data = scaler.fit_transform(train_data)
test_data = scaler.transform(test_data)

X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)

# Reshape input to be [samples, time steps, features] which is required for LSTM
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# Create and fit the LSTM network
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1))) # }
model.add(LSTM(units=50))
model.add(Dense(1))

model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=2) # Adjust epochs and b

# Make predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Invert predictions back to original scale
train_predict = scaler.inverse_transform(train_predict)
y_train = scaler.inverse_transform([y_train])
test_predict = scaler.inverse_transform(test_predict)
y_test = scaler.inverse_transform([y_test])

# Calculate root mean squared error
trainScore = np.sqrt(mean_squared_error(y_train[0], train_predict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = np.sqrt(mean_squared_error(y_test[0], test_predict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))

# Shift test predictions for 1-step
```

